ATC Developer Reference

ATC Developer Reference

This reference complements the Developer Guide. It focuses on practical contribution workflows for Advanced Trigonometry Calculator 2.1.9.

Main Project Structure

- `Advanced Trigonometry Calculator/`: C++ source code and Visual Studio project files.
- `tests/`: PowerShell regression, isolated coverage and memory stress tests.
- `docs/`: Markdown, PDF and DOCX documentation.
- `tools/docs/`: documentation generation helpers.

Coding Expectations

- Keep public command names stable unless a compatibility plan exists.
- Prefer existing parser, allocation and formatting helpers.
- Keep code comments in English.
- Avoid hidden behavior that cannot be tested or documented.
- Preserve Windows XP to Windows 11 compatibility goals where the current code supports them.

Adding a New Command

1. Locate the closest existing command handler in `commands.cpp`, `main_processor.cpp` or `main_aux_processor.cpp`. 2. Follow the existing command-detection style. 3. Keep prompts and outputs stable if they will be tested. 4. Add automated coverage in `tests/run-atc-regression.ps1` where possible. 5. Document the command in the user guide and coverage matrix.

Adding a Mathematical Function

1. Check whether the function belongs in `trigonometry.cpp`, `hyperbolic.cpp`, `logarithmic.cpp`, `statistics.cpp`, `digital_signal_processing.cpp` or another existing module. 2. Ensure both `double` and Boost `mp_float` paths are considered. 3. Confirm angle-mode behavior if the function is trigonometric. 4. Add autocomplete vocabulary if the function should be suggested. 5. Add regression tests with stable expected output.

Adding a Guided Module

1. Keep the interactive flow deterministic where practical. 2. Avoid opening external windows during automated tests. 3. Add at least a safe smoke test. 4. Document manual validation gaps if full automation is not practical.

Adding a Regression Test

Use `tests/run-atc-regression.ps1` for normal command/output checks. Keep tests:

- deterministic;
- independent of local absolute paths;
- safe under redirected input;
- isolated from user settings, or backed up/restored by the runner.

Do not update the documented test count unless the suite has been executed.

Documenting a Feature

When a feature changes:

1. update the English reference document first; 2. review the Portuguese counterpart; 3. update `tests/ATC_USER_GUIDE_COVERAGE.md`; 4. regenerate PDFs/DOCX when relevant.

Parser and Solver Regression Risks

Parser and solver changes should be tested with:

- direct arithmetic;
- implicit multiplication;
- variables;
- complex values;
- polynomial simplification;
- `solve equation(...)`;
- `solver(...)`;
- high precision mode where relevant.

Avoid solving a special case by bypassing the normal processing flow unless the behavior is deliberately documented and tested.

Windows Compatibility Notes

ATC contains console behavior for old and new Windows environments. Be careful with:

- Windows Terminal specific behavior;
- ANSI/VT color output;
- console window size and position;
- commands that launch new instances, tabs, files or folders.

Output Stability

Automated tests normally match output patterns. Before changing visible output, consider whether it is part of:

- user documentation;
- release notes;
- regression tests;
- exported text reports.

Pre-commit / Pre-release Checklist

- Build Release x64.
- Build Release x86 when compatibility-sensitive code changed.
- Run the regression suite.
- Run isolated coverage if the change touches interactive or source-level

behavior.

- Run memory stress tests for allocation-heavy changes.
- Update documentation and release notes.
- Check Markdown links and regenerate PDFs/DOCX when needed.